

8-Puzzle Solver

Table of Contents

- 1. [Introduction](#)
- 2. [Problem Description](#)
- 3. [Algorithms Implemented](#)
- 4. [Data Structures](#)
- 5. [Implementation Details](#)
- 6. [Sample Runs](#)

1. Introduction

This report documents the implementation of a comprehensive 8-puzzle solver using multiple search algorithms. The 8-puzzle is a sliding puzzle consisting of a 3x3 grid with 8 numbered tiles and one blank space. The objective is to reach a goal configuration from an initial state by sliding tiles into the blank space.

The implementation includes both uninformed search strategies (BFS, DFS, IDDFS) and informed search strategies (A* with Manhattan and Euclidean heuristics), along with a graphical user interface for visualization.

2. Problem Description

The 8-Puzzle Game

The 8-puzzle consists of:

- **Grid:** 3x3 board
- **Tiles:** Numbers 0-8, where 0 represents the blank space
- **Goal State:**

	1	2
3	4	5
6	7	8

Valid Moves

- **Up:** Slide a tile up into the blank space
- **Down:** Slide a tile down into the blank space
- **Left:** Slide a tile left into the blank space
- **Right:** Slide a tile right into the blank space

Solvability

A puzzle configuration is solvable if and only if the number of inversions is even. An inversion occurs when a tile with a higher number appears before a tile with a lower number (excluding the blank tile).

3. Algorithms Implemented

3.1 Breadth-First Search (BFS)

Description:

BFS explores the search space level by level, visiting all nodes at depth d before moving to depth $d+1$.

Algorithm Operation:

1. Initialize a queue with the initial state
2. Mark the initial state as visited
3. While the queue is not empty:
 - Dequeue the front node
 - If it's the goal state, return the solution
 - Generate all successor states
 - For each unvisited successor:
 - Mark it as visited
 - Enqueue it

Properties:

- **Complete:** Yes - always finds a solution if one exists
- **Optimal:** Yes - finds the shortest path
- **Time Complexity:** $O(b^d)$ where b is branching factor, d is depth
- **Space Complexity:** $O(b^d)$ - stores all nodes at current level

Data Structure: Queue (deque from collections module)

3.2 Depth-First Search (DFS)

Description:

DFS explores as far as possible along each branch before backtracking.

Algorithm Operation:

1. Start with the initial state
2. Mark the current state as visited
3. If the depth limit is reached or state is visited, return
4. If it's the goal state, return the solution
5. Recursively explore each successor

Properties:

- **Complete:** No - can get stuck in infinite loops (depth limit added)

- **Optimal:** No - may not find the shortest path
- **Time Complexity:** $O(b^m)$ where m is maximum depth
- **Space Complexity:** $O(bm)$ - only stores path from root to leaf

Data Structure: Recursion stack (implicit stack)

Note: Depth limit of 50 is implemented to prevent infinite loops.

3.3 Iterative Deepening Depth-First Search (IDDFS)

Description:

IDDFS combines the space efficiency of DFS with the optimality of BFS by running DFS with increasing depth limits.

Algorithm Operation:

1. For depth_limit from 0 to max_depth:
 - Run depth-limited DFS with current limit
 - If solution found, return it

Properties:

- **Complete:** Yes - finds solution if one exists
- **Optimal:** Yes - finds shortest path
- **Time Complexity:** $O(b^d)$ - similar to BFS
- **Space Complexity:** $O(bd)$ - better than BFS

Data Structure: Recursion stack with depth tracking

3.4 A* Search with Manhattan Distance Heuristic

Description:

A* uses a heuristic function to guide the search toward the goal. The evaluation function is $f(n) = g(n) + h(n)$, where:

- $g(n)$ = cost from start to node n
- $h(n)$ = estimated cost from n to goal (Manhattan distance)

Manhattan Distance Heuristic:

$$h(n) = \sum |current_row - goal_row| + |current_col - goal_col|$$

For each tile, calculate the sum of horizontal and vertical distances to its goal position.

Algorithm Operation:

1. Initialize priority queue with initial state (priority = $f(n)$)
2. While the priority queue is not empty:

- Pop node with lowest $f(n)$
- If it's the goal, return solution
- For each successor:
 - Calculate $g(n)$ and $h(n)$
 - Add to priority queue with priority $f(n) = g(n) + h(n)$

Properties:

- **Complete:** Yes
- **Optimal:** Yes (heuristic is admissible)
- **Time Complexity:** $O(b^d)$ but much faster in practice
- **Space Complexity:** $O(b^d)$

Data Structure: Priority queue (heapq module)

Admissibility: The Manhattan distance never overestimates the actual cost since:

- Each tile must move at least the Manhattan distance to reach its goal
- Multiple tiles cannot occupy the same space
- Therefore, $h(n) \leq h^*(n)$ where $h^*(n)$ is the true cost

3.5 A* Search with Euclidean Distance Heuristic

Description:

Similar to A* with Manhattan distance, but uses Euclidean (straight-line) distance as the heuristic.

Euclidean Distance Heuristic:

$$h(n) = \sum \sqrt{(\text{current_row} - \text{goal_row})^2 + (\text{current_col} - \text{goal_col})^2}$$

Properties:

- **Complete:** Yes
- **Optimal:** Yes (heuristic is admissible)
- **Performance:** Slightly less efficient than Manhattan for grid-based puzzles

Note: Euclidean distance is less informed than Manhattan distance for grid-based problems because it doesn't account for the constraint that tiles can only move horizontally or vertically.

4. Data Structures

4.1 GameNode Class

```
class GameNode:
    - config: Tuple[int, ...]      # Board configuration
    - prev_node: Optional[GameNode] # Parent node for path reconstruction
    - action: str                  # Move that led to this state
```

```
- level: int          # Depth in search tree
- empty_idx: int      # Position of blank tile
```

Purpose: Represents a state in the search space with backpointers for path reconstruction.

4.2 AlgorithmStats Class

```
@dataclass
class AlgorithmStats:
    - algo_label: str          # Algorithm name
    - visited_count: int       # Nodes expanded
    - max_depth: int          # Maximum search depth
    - solution_cost: int       # Path length
    - time_start: float        # Start timestamp
    - time_end: float          # End timestamp
    - action_sequence: List[str] # Solution path moves
    - node_sequence: List[GameNode] # Solution path states
    - found_solution: bool     # Success flag
```

Purpose: Stores performance metrics for comparison.

4.3 Data Structures by Algorithm

Algorithm	Primary Data Structure	Secondary
BFS	Queue (deque)	Set (visited nodes)
DFS	Recursion Stack	Set (visited nodes)
IDDFS	Recursion Stack	Set (visited nodes)
A* (Manhattan)	Priority Queue (heapq)	Set (visited nodes)
A* (Euclidean)	Priority Queue (heapq)	Set (visited nodes)

5. Implementation Details

5.1 Assumptions

- 1. **Valid Input:** The input configuration contains exactly 9 unique integers (0-8)
- 2. **Solvability Check:** Only solvable puzzles are processed (even inversions)
- 3. **Depth Limit:** DFS has a maximum depth of 50 to prevent infinite loops
- 4. **Goal State:** Fixed as (0, 1, 2, 3, 4, 5, 6, 7, 8)
- 5. **Move Ordering:** Consistent order (Up, Down, Left, Right) for reproducibility

5.2 Key Features

Solvability Detection:



```
def count_inversions(config: ConfigTuple) -> int:
    filtered = [x for x in config if x != 0]
    inv_count = 0
    for i in range(len(filtered)):
        for j in range(i + 1, len(filtered)):
            if filtered[i] > filtered[j]:
                inv_count += 1
    return inv_count

def check_solvability(config: ConfigTuple) -> bool:
    return count_inversions(config) % 2 == 0
```

Path Reconstruction:

```
def trace_back_path(target_node: GameNode) -> Tuple[List[str],
List[GameNode]]:
    actions, nodes = [], []
    current = target_node
    while current is not None:
        actions.append(current.action)
        nodes.append(current)
        current = current.prev_node
    actions.reverse()
    nodes.reverse()
    return actions, nodes
```

Successor Generation:

- For each state, generate valid moves based on blank tile position
- Corner positions: 2 possible moves
- Edge positions: 3 possible moves
- Center position: 4 possible moves

5.3 GUI Implementation

A graphical interface was implemented using Tkinter featuring:

- Interactive board input with preset configurations
- Algorithm selection
- Real-time visualization with step-by-step playback
- Results display with detailed metrics
- Solution tree visualization
- Scrollable interface for accessibility

6. Sample Runs

6.1 Easy Puzzle (5 Moves)

Initial Configuration:

1	2	5
3	4	8
6		7

Results:

- **BFS:** 5 nodes expanded, 3 moves, 0.000025s
 - **A* (Manhattan):** 4 nodes expanded, 3 moves, 0.000019s
-

6.2 Hard Puzzle (27 Moves) - Detailed Run

Initial Configuration:

8	6	7
2	5	4
3		1

Solvability Check:

- Inversions: 24 (even)
 - Solvable: Yes
-

BFS Results

Path to Goal: 27 steps

- Cost of Path: 27 moves
- Nodes Expanded: 167,760
- Search Depth: 27
- Running Time: 0.374 seconds

Solution Path:

Initial → Up → Right → Down → Left → Up → Up → Left → Down → Down
→ Right → Up → Up → Left → Down → Down → Right → Up → Right → Up
→ Left → Down → Right → Down → Left → Up → Left → Up

Analysis:

- BFS guarantees the optimal solution (27 moves)
 - High memory usage due to storing all nodes at each level
 - Expanded 167,760 nodes to find the solution
 - Suitable when optimality is required and memory is available
-

DFS Results

Path to Goal: N/A

- Cost of Path: 0 (no solution found within depth limit)
- Nodes Expanded: 116,770
- Search Depth: 50 (maximum)
- Running Time: 0.255 seconds

Analysis:

- DFS failed to find a solution within the depth limit of 50
 - Demonstrates DFS's incompleteness for this problem
 - Lower running time than BFS but no solution
 - Not recommended for 8-puzzle unless depth limit is higher
-

Iterative Deepening DFS Results

Path to Goal: 33 steps

- Cost of Path: 33 moves (not optimal)
- Nodes Expanded: 499,926
- Search Depth: 33
- Running Time: 1.056 seconds

Solution Path:

```
Initial → Up → Right → Down → Left → Up → Up → Right → Down → Down  
→ Left → Up → Up → Right → Down → Left → Up → Left → Down → Right  
→ Up → Left → Down → Down → Right → Up → Right → Down → Left → Up  
→ Right → Up → Left → Left
```

Analysis:

- Found a solution but not optimal (33 vs 27 moves)
 - Expanded the most nodes (499,926) due to repeated work
 - Complete and optimal in theory, but took longer path
 - High time complexity due to iterative nature
-

A* with Manhattan Distance Results

Path to Goal: 27 steps BEST PERFORMANCE

- Cost of Path: 27 moves (optimal)
- Nodes Expanded: 4,417
- Search Depth: 27
- Running Time: 0.020 seconds

Solution Path:

```
Initial → Up → Right → Down → Left → Up → Up → Left → Down → Down  
→ Right → Up → Up → Left → Down → Down → Right → Up → Right → Up  
→ Left → Down → Right → Down → Left → Up → Left → Up
```

Analysis:

- Found optimal solution (same as BFS)
- **97% fewer nodes expanded** compared to BFS (4,417 vs 167,760)
- **95% faster** than BFS (0.020s vs 0.374s)
- Most efficient algorithm for this problem
- Manhattan heuristic effectively guides search toward goal

Heuristic Performance:

- Average heuristic value: ~15-20 at start
- Heuristic reduces drastically as goal approaches
- Never overestimates (admissible)
- Consistent (satisfies triangle inequality)

A* with Euclidean Distance Results

Path to Goal: 27 steps

- Cost of Path: 27 moves (optimal)
- Nodes Expanded: 7,684
- Search Depth: 27
- Running Time: 0.036 seconds

Solution Path:

```
Initial → Up → Right → Down → Left → Up → Up → Left → Down → Down  
→ Right → Up → Up → Left → Down → Down → Right → Up → Right → Up  
→ Left → Down → Right → Down → Left → Up → Left → Up
```

Analysis:

- Found optimal solution (same as BFS and A* Manhattan)
- Expanded 74% more nodes than A* Manhattan (7,684 vs 4,417)
- Still 95% better than BFS
- Euclidean heuristic is less informed for grid-based movement
- Slight computational overhead from square root calculations

6.3 Comparison Table - Hard Puzzle

Algorithm Time (sec)	Success	Path Cost	Nodes Expanded	Search Depth
BFS 0.374	Yes	27	167,760	27
DFS 0.255	No	0	116,770	50
Iterative DFS 1.056	Yes	33	499,926	33
A* (Manhattan) 0.020	Yes	27	4,417	27
A* (Euclidean) 0.036	Yes	27	7,684	27